



Conception d'un prototype de drone par l'application des techniques plastronique

Équipe de projet

ADJLIA Katia

AOURTILANE Amazigh

KANE Rouguiyatou

SEBA Koussaila

GAOUA Imad-eddine

Année universitaire 2025–2026

Université Lyon 1 — Laboratoire Ampère

Responsable de projet : Philippe Lombard

Table des matières

I. Remerciements	4
II. Tableau de répartition des tâches par membre	4
III. Présentation du projet	5
1. Contexte général et problématique	5
2. Objectifs et critères de validation	6
3. Spécifications techniques principales	6
IV. État de l’art et positionnement	7
1. La carte de vol de référence : STEVAL-FCU001V2	7
2. Originalité de notre approche et différenciation	8
V. Analyse fonctionnelle du système	8
1. Décomposition en sous-systèmes	8
2. Tableau des 7 blocs fonctionnels	11
3. Mapping des broches STM32 — affectation verrouillée	12
VI. Justification de la plastronique 3D	12
1. Principe de la technologie IME	12
2. Avantages de la plastronique et du thermoformage	13
3. Contraintes techniques à respecter	13
VII. Bill of Materials (BOM)	13
VIII. Schématique KiCad	15

1. Description générale du schéma final	15
2. Choix des empreintes	15
3. Contraintes rencontrées et solutions apportées	16
4. Tableau de validation des 7 blocs	17
IX. Conception du modèle 3D	20
1. Paramètres généraux de conception	21
2. Conception du moule du châssis	21
3. Conception des supports moteurs et protège-hélices	22
X. Firmware STM32 — Architecture logicielle	23
1. Environnement de développement et base de code	23
2. Structure du firmware	23
3. Fonctionnement général du firmware	24
4. Adaptation des paramètres	25
XI. Tests à réaliser sur le prototype	26
1. LED : soudure et test	26
XII. Conclusion et perspectives	26
XIII. Annexes	28
Annexe A — Schéma KiCad complet	28
Annexe B — Code firmware complet	29
1. main.c — Boucle principale à 800 Hz	29
2. motor.c — Commande PWM des 4 moteurs	30

3. LSM6DSL_ACC_GYRO_driver.c — Lecture registre IMU via SPI	31
4. bluenrg_gap_aci.c — Initialisation BLE	31
5. flight_control.c — PID et calcul PWM moteurs	32
6. ahrs.c — Fusion Mahony	34
7. debug.c — Sortie UART 115200	35
Annexe C — Circuits porte-clefs individuels	36
Annexe D — Références techniques	42 .

I. Remerciement

Nous remercions Monsieur Philippe Lombard pour le suivi réalisé durant ce projet ainsi que pour les conseils apportés au cours de sa réalisation.

En effet, ce projet de drone n'a pas été simple à réaliser. Nous avons rencontré plusieurs difficultés dès le début, nous avons eu du mal à comprendre les tâches à effectuer, lors de la conception du système, à l'intégration des différents composants électroniques. Ces obstacles nous ont toutefois permis de développer notre capacité d'adaptation, notre esprit d'analyse et notre travail en équipe afin de trouver des solutions efficaces.

Nous remercions également l'ensemble de l'équipe pédagogique pour les enseignements dispensés durant cette formation, qui nous ont permis d'acquérir les connaissances nécessaires à la réalisation de ce projet, notamment dans les domaines de l'électronique, des systèmes embarqués et de la conception mécanique.

Enfin, nous adressons nos remerciements à nos camarades ainsi qu'à nos proches pour leurs échanges constructifs, leurs encouragements et leur soutien tout au long de ce projet.

II. Tableau de répartition des tâches par membre

Le tableau ci-après identifie la contribution de chaque membre de l'équipe pour l'ensemble des tâches réalisées au cours du projet. Trois niveaux de participation sont distingués :

- ☒☒☒ : désigne le membre qui a assuré la totalité du travail pour la tâche concernée.
- ☒☒ : indique qu'il en a assuré la majorité.
- ☒ : signifie qu'il y a participé de manière complémentaire.

Taches	ADJLIA	AOURTILANE	KANE	SEBA	GAOUA
Cahier des charges	☒☒	☒	☒☒	☒☒	☒
Schéma KiCad		☒		☒☒☒	
Développement logiciel & firmware	☒☒☒		☒☒☒		
Modélisation et Impression en 3D		☒☒☒			☒☒
Analyse et maîtrise des contraintes mécaniques		☒		☒	☒☒☒
Assemblage & tests	☒☒	☒☒	☒☒	☒☒	☒☒
Rapport final	☒☒	☒☒	☒☒	☒☒	☒☒

III. Présentation du projet

1. Contexte général et problématique

Les drones grand public reposent aujourd'hui sur des architectures électroniques classiques, où le circuit imprimé est une pièce distincte du châssis, reliée à la structure

par des nappes de câblage et des fixations mécaniques. Cette approche engendre une masse supplémentaire, une vulnérabilité aux vibrations aux points de connexion, et une complexité d'assemblage significative. La plastronique 3D offre une réponse directe à ces limitations en fusionnant la fonction électronique et la structure mécanique en un seul et même objet.

C'est dans ce contexte que s'inscrit notre projet : concevoir et réaliser un quadricoptère dont la carte de vol est intégrée directement au châssis par la technique IME (In-Mold Electronics). Cette technologie consiste à déposer des encres conductrices sur un substrat thermoplastique avant sa mise en forme par thermoformage, produisant ainsi un circuit imprimé tridimensionnel porté par la pièce structurelle elle-même. L'objectif est de valider la faisabilité de cette approche sur un système aérien réel, soumis à des contraintes mécaniques et électriques simultanées.



Résultat attendu sur le drone final

2. Objectifs et critères de validation

Le prototype doit satisfaire à un ensemble de critères de validation répartis en quatre catégories.

- Sur le plan du vol, le drone doit être capable d'un vol stable et contrôlé via l'application mobile ST_BLE_DRONE, avec une réponse aux commandes de roulis, tangage et lacet conforme aux ordres envoyés par l'utilisateur.

- Sur le plan mécanique, le châssis imprimé en 3D doit supporter sans déformation permanente les contraintes vibratoires générées par les quatre moteurs en fonctionnement continu.
- Sur le plan électrique, l'intégrité des pistes IME doit être maintenue sous charge réelle, c'est-à-dire avec un courant total de l'ordre de 2.6 A circulant dans les conducteurs.
- Sur le plan dimensionnel, le drone doit respecter un gabarit maximal de 20 × 20 cm — correspondant au format A4 — et une masse inférieure à 250 g.

3. Spécifications techniques principales

Le tableau suivant récapitule les spécifications techniques retenues, issues de l'analyse du kit STEVAL-DRONE02 de STMicroelectronics et des datasheets des composants sélectionnés.

Paramètres	Valeur retenue	Source
Microcontrôleur	STM32F401CCU6 — 84 MHz, Cortex-M4, QFN-48	STMicroelectronics
Module Bluetooth	BlueNRG-M0A — BLE 4.2, SPI, LCC-11	STMicroelectronics
Centrale inertielle (IMU)	LSM6DSR — accéléromètre + gyroscope 3 axes, LGA-14	STMicroelectronics
Batterie	LiPo 1 cellule, 3.7 V, 600 mAh, 30C max	Kit STEVAL-DRONE02
Moteurs	DC coreless 3.7 V, dimensions 85 × 20 mm, 4 unités	Kit STEVAL-DRONE02
Hélices	Diamètre 65 mm — 2 dans le sens horaire, 2 antihoraire	Kit STEVAL-DRONE02
Autonomie théorique	13.8 min (600 mAh ÷ 2.6 A)	Calcul équipe
Autonomie réelle estimée	Objectif 5 minutes en conditions de vol réelles	Calcul équipe
Gabarit maximal du châssis	20 × 20 cm (format A4)	Contrainte projet
Masse cible totale	< 250 g (châssis + électronique + batterie)	Contrainte projet

IV. État de l'art et positionnement

1. La carte de vol de référence : STEVAL-FCU001V2

STMicroelectronics a développé la carte STEVAL-FCU001V2, une carte de vol pour quadricoptère de petite taille qui sert de référence technique directe à notre projet. Cette carte intègre sur un PCB FR4 rigide classique de format compact les composants suivants : un microcontrôleur STM32F401CCU6, un module Bluetooth basse consommation BlueNRG-M0A, une centrale inertielle LSM6DSR, un capteur barométrique LPS22HH pour l'estimation d'altitude, un chargeur de batterie LiPo STC4054 alimenté par USB, et quatre MOSFET STL10N3LLH5 permettant le pilotage direct de moteurs DC à balais. La carte peut être commandée par l'application mobile ST_BLE_DRONE, disponible sur Android et iOS, qui transmet les commandes de vol via la liaison Bluetooth 4.2.

Cette carte constitue un référentiel validé et certifié — elle a passé les certifications FCC, IC et CE — sur lequel nous appuyons notre conception pour garantir la compatibilité logicielle avec le firmware de référence STSW-FCU001, lui-même open source sur GitHub.

2. Originalité de notre approche et différenciation

Notre projet se différencie de la carte de référence sur trois axes majeurs. Premièrement, l'intégration plastronique 3D par technologie IME constitue la rupture fondamentale : les pistes électriques ne sont pas gravées sur un PCB séparé mais moulées directement dans la structure thermoplastique du châssis. Cette fusion élimine les nappes de câblage et les connecteurs intermédiaires, réduisant simultanément la masse et les points de défaillance mécaniques.

Deuxièmement, notre version V1 adopte une architecture à quatre sorties ESC externes plutôt que des MOSFET intégrés. Ce choix ouvre la possibilité d'utiliser des moteurs brushless, plus efficaces énergétiquement que les moteurs à balais de la carte ST, et simplifie la conception thermique en reportant la dissipation de puissance vers les contrôleurs de vitesse externes.

Troisièmement, l'architecture logicielle distingue clairement deux bus SPI indépendants : SPI1 dédié au module BlueNRG-M0A et SPI2 dédié à l'IMU LSM6DSR. Cette séparation améliore l'isolation électrique entre les deux périphériques, réduit la latence de lecture des données inertielles, et simplifie le développement du firmware en évitant tout arbitrage de bus.

V. Analyse fonctionnelle du système

1. Décomposition en sous-systèmes

Pour concevoir notre drone, nous avons divisé l'électronique en sept parties distinctes, que nous appelons **blocs fonctionnels**. Chaque bloc a un rôle précis et ne s'occupe que de sa fonction. Cette façon de travailler présente deux avantages concrets : d'abord, si quelque chose ne fonctionne pas, on sait immédiatement dans quel bloc

chercher le problème. Ensuite, si on veut améliorer une fonctionnalité dans une version future, on modifie uniquement le bloc concerné sans toucher au reste.

Bloc 1 — Le microcontrôleur (MCU) : le cerveau du drone

Le premier bloc est le plus important de toute la carte. Il contient le microcontrôleur **STM32F401CCU6**, qui est en réalité un petit ordinateur intégré dans une puce de 7 mm × 7 mm. C'est lui qui prend toutes les décisions : il lit les capteurs, calcule comment stabiliser le drone, envoie les ordres aux moteurs et communique avec le téléphone via Bluetooth.

Pour que ce microcontrôleur fonctionne correctement, il a besoin de plusieurs éléments autour de lui. Le **quartz 16 MHz** est son horloge — sans lui, le microcontrôleur ne saurait pas à quelle vitesse travailler. Ce quartz est accompagné de deux condensateurs de 15 pF qui permettent à l'oscillation d'être stable et précise. Le **condensateur VCAP1 de 2.2 µF** est connecté sur une broche spéciale du STM32 qui alimente son régulateur interne de tension : sans ce condensateur, le microcontrôleur peut démarrer de façon aléatoire ou se bloquer. La **résistance de 10 kΩ sur NRST** maintient le microcontrôleur dans un état de fonctionnement normal en permanence — si cette résistance est absente, la broche de reset peut capter du bruit électrique et redémarrer le microcontrôleur involontairement. La **résistance de 10 kΩ sur BOOT0** force le microcontrôleur à démarrer depuis sa mémoire interne et non depuis un port de programmation — c'est indispensable pour que le drone vole normalement. Enfin, les **condensateurs de découplage de 100 nF** placés sur chaque broche d'alimentation filtrent les petites variations de tension qui apparaissent quand le microcontrôleur travaille intensément.

Bloc 2 — Le port SWD : la porte d'entrée du programmeur

Le deuxième bloc est un simple **connecteur de 5 broches** (2.54 mm) qui sert exclusivement à programmer et déboguer le microcontrôleur. SWD signifie *Serial Wire Debug* — c'est le protocole utilisé par tous les outils de développement STM32.

Concrètement, on branche un câble entre ce connecteur et un appareil appelé **ST-LINK** (fourni avec la carte de développement ou acheté séparément), lui-même connecté à l'ordinateur par USB. Depuis le logiciel **STM32CubeIDE**, on peut alors charger le programme dans la mémoire du microcontrôleur, l'exécuter pas à pas, et lire la valeur de chaque variable en temps réel pour comprendre ce qui se passe à l'intérieur. Les 5 broches du connecteur sont : l'alimentation +3.3 V, le signal SWDIO (données), le signal SWCLK (horloge), le signal NRST (reset) et la masse GND. Sans ce bloc, il est impossible de mettre du code dans le drone.

Bloc 3 — Le port UART : la fenêtre de débogage

Le troisième bloc est un **connecteur de 4 broches** (2.54 mm) qui permet d'afficher des messages texte depuis le microcontrôleur vers un ordinateur. UART signifie *Universal*

Asynchronous Receiver Transmitter — c'est le protocole de communication série le plus simple qui existe en électronique embarquée.

En pratique, pendant le développement du firmware, le programmeur insère dans le code des lignes du type `printf("Vitesse moteur 1 : %d\n", vitesse)`. Ces messages sont envoyés à 115 200 bits par seconde via ce connecteur vers un adaptateur USB-UART branché à l'ordinateur. Sur l'écran apparaît alors un terminal qui affiche en temps réel toutes les informations : tension de la batterie, angle du drone mesuré par l'IMU, commandes reçues via Bluetooth. Sans ce bloc, déboguer le firmware serait extrêmement difficile car on ne pourrait pas voir ce qui se passe à l'intérieur du microcontrôleur pendant le vol.

Bloc 4 — Le module Bluetooth : le lien avec le pilote

Le quatrième bloc contient le module **BlueNRG-M0A** de STMicroelectronics, qui assure toute la communication sans fil entre le drone et le smartphone du pilote. Ce module implémente le protocole **Bluetooth Low Energy 4.2** — une version économe en énergie du Bluetooth, conçue spécifiquement pour les objets connectés embarqués sur batterie.

Le module communique avec le microcontrôleur via le bus **SPI1** (broche PA5 pour l'horloge, PA6 pour les données entrant, PA7 pour les données sortant, PA4 pour la sélection du module). Deux signaux de contrôle supplémentaires sont nécessaires : **BLE_IRQ** sur PB0, que le module utilise pour signaler au microcontrôleur qu'un message est arrivé du téléphone, et **BLE_RSTN** sur PB1, qui permet de réinitialiser le module si besoin. La résistance de **47 kΩ sur BLE_IRQ** est imposée par le fabricant dans la fiche technique du composant : sans elle, cette broche flotte à un niveau de tension indéterminé quand le module est en veille, et le microcontrôleur reçoit de fausses alertes en permanence. Les condensateurs de **100 nF et 1 µF** directement au niveau du module filtrent les variations d'alimentation qui surviennent quand il émet en radio.

Bloc 5 — La centrale inertielle (IMU) : les yeux du drone

Le cinquième bloc contient le capteur **LSM6DSR** de STMicroelectronics. IMU signifie *Inertial Measurement Unit* — en français, centrale inertielle. Ce composant mesure en permanence deux grandeurs physiques : l'**accélération** sur les trois axes de l'espace (avant/arrière, gauche/droite, haut/bas) grâce à un accéléromètre, et la **vitesse de rotation** sur les trois axes grâce à un gyroscope. Ces mesures permettent au firmware de calculer en temps réel l'orientation du drone — s'il penche à gauche, s'il tourne, s'il monte ou descend — et d'envoyer les corrections nécessaires aux moteurs pour le stabiliser.

Le LSM6DSR communique avec le microcontrôleur via le bus **SPI2** (PB13, PB14, PB15, PB12), séparé du bus BLE pour éviter tout conflit. La broche **INT1** (connectée sur PB10) est une sortie d'interruption : quand le capteur a une nouvelle mesure disponible, il envoie un signal sur cette broche pour prévenir immédiatement le microcontrôleur, sans que celui-ci soit obligé de demander les données en permanence. Les condensateurs de **100**

nF et 1 μ F protègent l'alimentation du capteur des perturbations générées par les moteurs.

Bloc 6 — L'alimentation : le cœur énergétique

Le sixième bloc gère toute l'énergie de la carte. La batterie **LiPo 1S** fournit une tension variable entre 3.2 V (déchargée) et 4.2 V (pleine charge), connectée via un **connecteur JST PH 2 broches**. Or le microcontrôleur, le module BLE et l'IMU fonctionnent tous à **3.3 V exactement** — une tension plus haute les endommagerait. Le **régulateur LDO LD39015M33R** (boîtier SOT-23-5) reçoit la tension batterie en entrée et délivre un 3.3 V parfaitement stable en sortie, quelle que soit l'état de charge de la batterie. Les deux condensateurs de **4.7 μ F** en entrée et en sortie du LDO stabilisent la régulation et évitent les oscillations.

Le **pont diviseur résistif** formé par R4 (100 k Ω) et R2 (47 k Ω) résout un problème précis : l'entrée ADC du microcontrôleur ne peut pas lire une tension supérieure à 3.3 V, mais la batterie peut monter à 4.2 V. Le pont divise la tension de la batterie dans un rapport de 47/147, ce qui donne au maximum $4.2 \times 47/147 = \mathbf{1.34\ V}$ — une valeur compatible avec l'ADC. Le condensateur de 100 nF sur ce nœud filtre les parasites haute fréquence venant des ESC. Grâce à cette mesure, le firmware peut surveiller en permanence l'état de la batterie et prévenir le pilote quand elle est presque vide.

Bloc 7 — Les sorties ESC : le lien avec les moteurs

Le septième et dernier bloc regroupe les quatre **connecteurs PinHeader 1×3** (J_ESC1 à J_ESC4) qui transmettent les ordres de vitesse aux ESC externes. ESC signifie *Electronic Speed Controller* — ce sont des petits modules électroniques qui reçoivent un signal de commande et alimentent les moteurs en conséquence.

Le signal de commande standard utilisé par tous les ESC du marché est un **signal PWM à 50 Hz** : une impulsion répétée toutes les 20 ms, dont la durée varie entre 1 ms (moteur à l'arrêt) et 2 ms (plein régime). Ce signal est généré par le **Timer 4** du STM32F401, configuré avec un prescaler de 83 et un compteur maximum de 19 999, ce qui produit exactement une période de 20 ms. Chaque connecteur comporte 3 broches : la broche de signal PWM, une masse GND commune (indispensable comme référence pour que le signal soit interprété correctement par l'ESC), et la tension batterie VBAT. Les **résistances de 100 Ω** en série sur chaque signal protègent les broches du microcontrôleur : si un câble est court-circuité ou si l'ESC présente un défaut, le courant traversant la résistance reste limité à un niveau inoffensif pour la puce.

1. Tableau des 7 blocs fonctionnels

Bloc	Composants principaux	Fonction assurée
Bloc 1 — MCU	STM32F401CCU6, Y1 16 MHz, VCAP1 2.2µF, R NRST, R BOOT0, condensateurs découpl.	Exécution du firmware, génération PWM, lecture capteurs, mesure ADC
Bloc 2 — SWD	Connecteur P2 5 broches (2.54 mm)	Programmation et débogage via ST-LINK/V2 ou V3
Bloc 3 — UART	Connecteur P1 4 broches (2.54 mm)	Console de débogage série à 115 200 bauds
Bloc 4 — Bluetooth	BlueNRG-M0A, 100 nF + 1 µF découpl., R pull-down IRQ 47 kΩ	Communication BLE 4.2 avec l'application ST_BLE_DRONE
Bloc 5 — IMU	LSM6DSR, 100 nF + 1 µF découpl., liaison INT1	Mesure accélération et vitesse angulaire sur 3 axes
Bloc 6 — Alimentation	LDO 3.3 V SOT-23-5, J1 JST batterie, pont R 100k/47k, C 100 nF	Régulation 3.3 V, mesure tension batterie via PA0 (ADC)
Bloc 7 — ESC	4 connecteurs 1×3 bro., 4 résistances série 100 Ω	Transmission des signaux PWM 50 Hz vers les 4 ESC externes

2. Mapping des broches STM32 — affectation verrouillée

L'affectation des broches du STM32F401CCU6 a été définie conformément à l'architecture de la carte de référence STEVAL-FCU001V2, afin de garantir la compatibilité directe avec le firmware open source STSW-FCU001. Ce mapping est considéré comme verrouillé et ne doit pas être modifié sans régénérer l'intégralité de la configuration HAL dans STM32CubeIDE.

Bus / Fonction	Broche(s) STM32	Signal(s) associé(s)
SPI1 — BlueNRG (horloge, données)	PA5, PA6, PA7, PA4	BLE_SCK, BLE_MISO, BLE_MOSI, BLE_CS
SPI1 — BlueNRG (contrôle)	PB0, PB1	BLE_IRQ, BLE_RSTN
SPI2 — LSM6DSR (horloge, données)	PB13, PB14, PB15, PB12	IMU_SCK, IMU_MISO, IMU_MOSI, IMU_CS
Interruption IMU	PB10	IMU_INT1
TIM4 — Sorties PWM ESC	PB6, PB7, PB8, PB9	M1_PWM, M2_PWM, M3_PWM, M4_PWM
USART1 — Debug série	PA9, PA10	UART_TX, UART_RX
SWD — Programmation	PA13, PA14	SWDIO, SWCLK

Bus / Fonction	Broche(s) STM32	Signal(s) associé(s)
ADC12 — Mesure batterie	PA0	VBAT_SENSE

VI. Justification de la plastronique 3D

1. Principe de la technologie IME

La technologie In-Mold Electronics (IME) est une méthode de fabrication qui consiste à imprimer des encres conductrices par sérigraphie ou jet d'encre sur une feuille de substrat thermoplastique, puis à mettre en forme ce substrat par thermoformage à chaud pour obtenir une pièce tridimensionnelle aux fonctions électriques intégrées. Les composants électroniques sont ensuite reportés en surface par brasage, puis la pièce peut être surmoulée par injection pour l'encapsulation finale.

Appliquée au châssis d'un drone, cette technique transforme chaque bras de la structure en un conducteur actif : les pistes d'alimentation des moteurs et les signaux de commande des ESC circulent dans l'épaisseur même du thermoplastique, sans fil ni connecteur intermédiaire. Le résultat est un objet qui est à la fois la structure porteuse et le circuit imprimé.

2. Avantages de la plastronique et thermoformage

La **plastronique 3D** associée au **thermoformage** permet d'intégrer directement les pistes conductrices dans le châssis du drone. Cette solution rend le système **plus compact**, car elle réduit le nombre de câbles, de connecteurs et de pièces séparées. Elle permet aussi de diminuer le **coût de fabrication** en limitant les étapes d'assemblage et les risques d'erreur, surtout en production série. C'est également une technologie plus **moderne**, car elle combine la mécanique, l'électronique et le design dans une seule structure.

Enfin, cette approche peut être plus **responsable et éthique**, car elle réduit l'utilisation de matériaux inutiles, limite le gaspillage et favorise une conception plus légère et mieux optimisée. Ainsi, la plastronique et le thermoformage offrent une solution **plus légère, plus compacte, plus fiable et mieux adaptée** à un drone miniature.

2. Contraintes techniques à respecter

La conception IME impose des contraintes géométriques et matériaux précises. Le rayon de courbure minimal des pistes conductrices doit être supérieur ou égal à 3 mm pour éviter toute rupture de la métallisation lors du thermoformage. La largeur de piste minimale est fixée à 0.5 mm pour les signaux numériques (PWM, SPI) et à 1 mm pour les pistes d'alimentation soumises à un courant continu supérieur à 1 A. Le matériau du châssis doit être un thermoplastique compatible thermoformage, avec une résistivité électrique supérieure à $10^{12} \Omega \cdot \text{cm}$ pour garantir l'isolation entre pistes adjacentes. Les matériaux ABS et polycarbonate (PC) sont les candidats retenus pour la V1.

VII. Bill of Materials (BOM)

La nomenclature suivante liste l'ensemble des composants du schéma KiCad validé. Les références sont celles du schéma final (fichier schema1_FIN.kicad_sch). Les empreintes correspondent aux boîtiers CMS retenus pour le placement et le routage PCB. Tous les composants passifs sont en boîtier 0805 afin de faciliter le brasage manuel sur le prototype.

Référence	Composant	Valeur / Réf. fabricant	Qte	Boîtier	Fonction schéma
U2	Microcontrôleur	STM32F401CCU6	1	QFN-48 7×7 mm	Cerveau système
U3	Module BLE	BlueNRG-M0A	1	LCC-11	Connexion Bluetooth
U4	Centrale inertielle	LSM6DSR	1	LGA-14 3×2.5 mm	Accéléromètre / gyroscope
U1	Régulateur LDO	LD39015M33R	1	SOT-23-5	Régulation 150 mA
Y1	Quartz	NDK NX2520SA-16MHz	1	SMD 2520-4Pin	Horloge STM32 16 MHz
C1, C4	Condensateur	4.7 μ F, Céram. X5R 16V	2	0805	Filtrage entrée / sortie
C2, C3	Condensateur	15 pF, Céram. C0G 50V	2	0805	Charge Y1
C5	Condensateur	100 nF, Céram. X7R 16V	1	0805	Filtrage VBAT
C6	Condensateur	2.2 μ F, Céram. X5R 16V	1	0805	VCAP1 régulateur STM32
C7	Condensateur	1 μ F, Céram. X5R 16V	1	0805	Découplage global
C8, C9, C10	Condensateur	100 nF, Céram. X7R 16V	3	0805	Découplage BLE, IMU
C9-IMU	Condensateur	1 μ F, Céram. X5R 16V	1	0805	Découplage LSM6DSR
R4	Résistance	100 k Ω , 1%, 0.1 W	1	0805	Bras haut VBAT
R2	Résistance	47 k Ω , 1%, 0.1 W	1	0805	Bras bas VBAT

Référence	Composant	Valeur / Réf. fabricant	Qte	Boîtier	Fonction schéma
R3	Résistance	47 kΩ, 1%, 0.1 W	1	0805	Pull-down BLE_IP (impos datash
R5, R6	Résistance	2.2 kΩ, 1%, 0.1 W	2	0805	Limitati LED LD
R_M1 à R_M4	Résistance	100 Ω, 1%, 0.1 W	4	0805	Protect signaux
Rboot1	Résistance	10 kΩ, 1%, 0.1 W	1	0805	Pull-down BOOTC
SW1	Bouton poussoir	Tactile SMD, 4.2×3.2 mm	1	SW_SPST_CK_RS282G05A3	Reset n NRST
LD1, LD2	LED	Rouge, 20 mA, SMD	2	LED 0805	Témoin aliment BLE ac
J1	Connecteur	JST PH 2 broches, 2 mm	1	JST PH B2B-PH-K	Connex batterie
J_ESC1 à J_ESC4	Connecteur	PinHeader 1×3, 2.54 mm	4	PinHeader 1x03	Sorties signal/ vers ES
P1	Connecteur	PinHeader 1×4, 2.54 mm	1	PinHeader 1x04	Debug (TX, RX, GND)
P2	Connecteur	PinHeader 1×5, 2.54 mm	1	PinHeader 1x05	Port SW (SWDI, SWCLK, +3V3, GND)

VIII. Schématique Kicad

1 Description générale du schéma final

Le schéma final a été réalisé sous KiCad 9.0.1. Il se compose d'une feuille unique de format A4, organisée selon une disposition spatiale cohérente : le bloc d'alimentation est positionné à gauche, le microcontrôleur au centre, les blocs BLE et IMU à droite, et les sorties ESC ainsi que les connecteurs de debug en périphérie. Cette organisation reflète le flux d'information du système, de la source d'énergie vers les actionneurs.

L'ensemble des empreintes KiCad est assigné et l'export vers le PCB se termine avec zéro erreur et zéro avertissement. Le rapport ERC (Electrical Rules Check) interne ne signale aucune violation électrique. La vérification des nets de masse — qui avait révélé cinq connexions erronées GND/+3V3 lors du développement — a été menée pin par pin et toutes les anomalies ont été corrigées.

2 Choix des empreintes

Le choix des empreintes a été une étape importante dans la conception du schéma, car une mauvaise empreinte peut rendre le routage PCB impossible ou provoquer des courts-circuits entre les couches de cuivre.

Pour tous les composants passifs du schéma résistances de tirage, condensateurs de découplage, condensateurs de filtrage, résistances de limitation de courant pour les LED et résistances série sur les signaux ESC nous avons utilisé le boîtier **CMS 0805** (dimensions 2.0 mm × 1.25 mm). Ce boîtier est suffisamment grand pour être brasé manuellement à la panne fine, et il est disponible nativement dans les bibliothèques KiCad sous les références

Resistor_SMD: R_0805_2012Metric

Capacitor_SMD:C_0805_2012Metric.

En revanche, pour les **résistances du pont diviseur VBAT_SENSE**, c'est-à-dire R4 (100 kΩ) et R2 (47 kΩ), nous avons choisi le boîtier **CMS 1206** (3.2 mm × 1.6 mm). Ce choix a été fait pour une raison précise liée au routage : le boîtier 1206 est plus grand que le 0805, ce qui donne des pastilles de cuivre plus espacées sur le PCB. Cet espacement supplémentaire garantit que les couches de cuivre voisines pistes, pastilles et plans de masse ne se touchent pas lors du routage, et que les règles DRC (Design Rule Check) de KiCad sont respectées. Sur une carte comme la nôtre, toutes les zones conductrices doivent être séparées par une distance minimale pour éviter les courts-circuits ; utiliser le 1206 sur ces deux résistances critiques permet d'atteindre cet objectif sans contrainte supplémentaire lors du routage.

3 Contraintes rencontrées et solutions apportées

La réalisation du schéma sous KiCad n'a pas été immédiate. Plusieurs obstacles techniques ont été identifiés et résolus au cours du développement.

Problème 1 — Routage monocouche En plastronique IME, on ne peut utiliser qu'une seule couche de cuivre. Impossible donc de faire passer une piste sous une autre comme sur un PCB classique. **Solution** : utilisation de résistances de pont à 0 Ω aux croisements inévitables, et placement des composants optimisé pour minimiser ces croisements.

Problème 2 — Pas de plan de masse GND Sur un PCB traditionnel, toute une couche est dédiée à la masse — c'est ce qu'on appelle un plan de masse. En monocouche, c'est impossible. Sans lui, les alimentations sont moins stables et les signaux plus sensibles aux interférences. **Solution** : toutes les pistes GND sont aussi courtes et larges que possible (1 mm minimum), et les condensateurs de découplage du STM32 convergent vers un point de masse commun unique — technique du *star grounding*.

Problème 3 — Pistes de puissance trop proches des pistes de signal Les pistes VBAT qui alimentent les ESC transportent plusieurs ampères, tandis que les pistes SPI transportent des signaux faibles à plusieurs mégahertz. Si elles sont trop proches, les variations de courant moteur injectent du bruit dans les signaux. **Solution** : séparation minimale de 1.5 mm imposée entre pistes de puissance et pistes de signal.

Problème 4 — Rayon de courbure des pistes IME Lors du thermoformage du châssis, les pistes sont déformées avec le plastique. Un angle trop serré provoque une rupture de la métallisation invisible à l'œil. **Solution** : rayon de courbure minimal de 3 mm respecté sur tous les virages de pistes, angles droits interdits.

Problème 5 — Zone d'exclusion autour de l'antenne BLE Le module BlueNRG-M0A possède une antenne céramique interne. Toute piste de cuivre trop proche perturbe son rayonnement et réduit la portée Bluetooth. **Solution** : zone d'exclusion de cuivre de 6 mm autour de l'antenne, définie et vérifiée automatiquement par le DRC de KiCad.

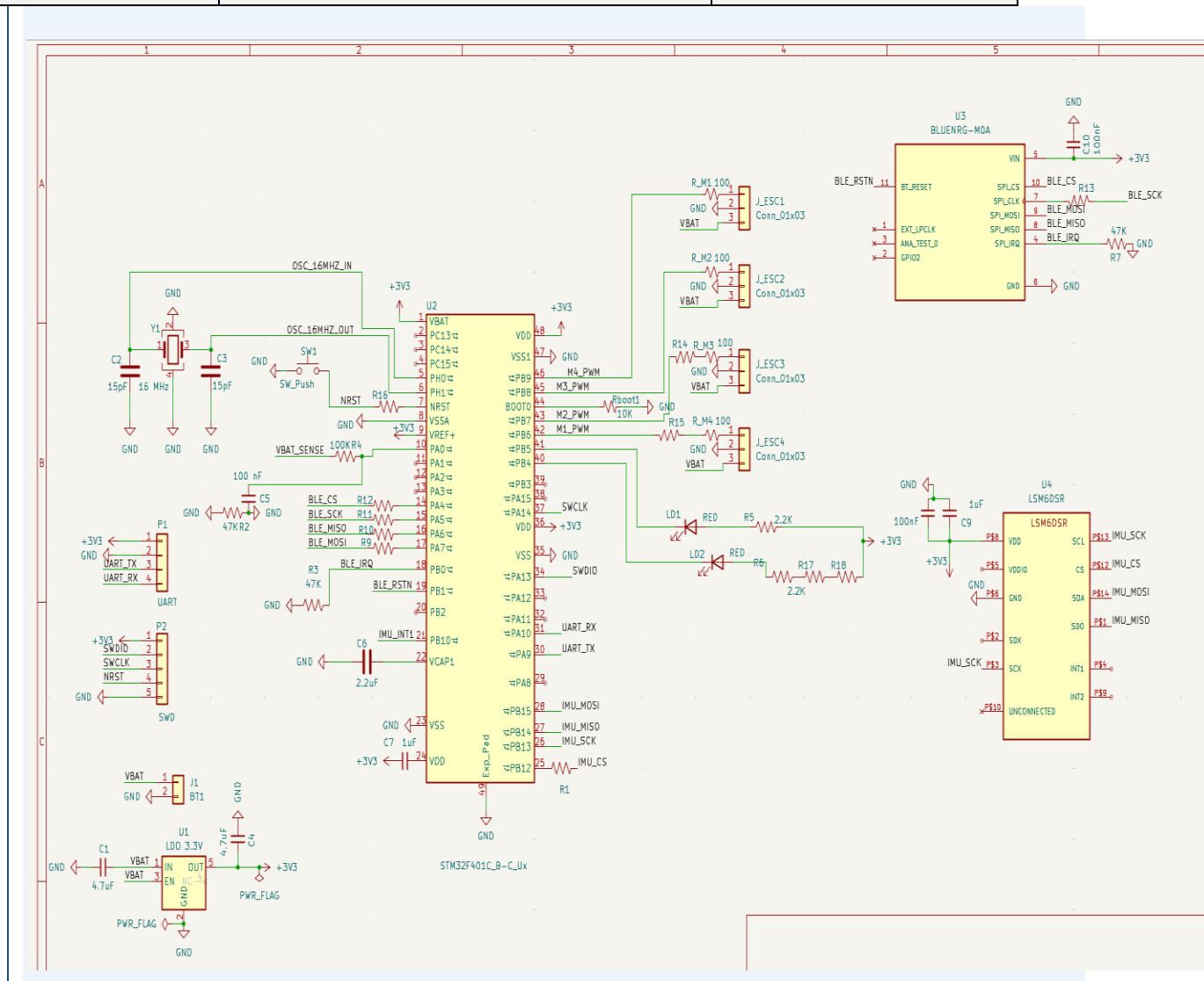
Problème 6 — Composants absents de la bibliothèque KiCad Le BlueNRG-M0A et le LSM6DSR ne sont pas dans la bibliothèque standard de KiCad 9.0.1. **Solution** : symboles et empreintes téléchargés depuis le site SnapMagic et installés manuellement dans KiCad via le gestionnaire de bibliothèques.

Problème 7 — Résistance R1 sans empreinte persistante Malgré plusieurs corrections dans les propriétés du composant, l'empreinte de R1 n'était pas sauvegardée et l'export PCB continuait à signaler une erreur. **Solution** : fermeture complète de KiCad, réouverture du seul fichier schéma, correction de l'empreinte, sauvegarde Ctrl+S, puis relance de l'export — erreur définitivement corrigée.

4 Tableau de validation des 7 blocs

Bloc	Composants clefs du schéma	Statut de validation
MCU (U2)	STM32F401CCU6 QFN-48, Y1 16 MHz, C6 2.2 μ F VCAP1, SW1, Rboot1 10k	Validé — 0 erreur ERC
SWD (P2)	5 broches : +3V3, SWDIO, SWCLK, NRST, GND	Validé

Bloc	Composants clefs du schéma	Statut de validation
UART (P1)	4 broches : +3V3, GND, UART_TX (PA9), UART_RX (PA10)	Validé
Bluetooth (U3)	BlueNRG-M0A, C8 100nF, 1µF découpl., R3 47k pull-down sur IRQ	Validé
IMU (U4)	LSM6DSR LGA-14, 100nF + 1µF découpl., INT1 connecté sur PB10	Validé
Alimentation (U1)	LDO SOT-23-5, J1 JST, C1/C4 4.7µF, pont R4 100k / R2 47k / C5 100nF	Validé
ESC (J_ESC1-4)	4 connecteurs 1×3, R_M1-R_M4 100Ω en série, VBAT sur pin 3	Validé



Mise à jour du PCB à partir du Schéma ✕

Options

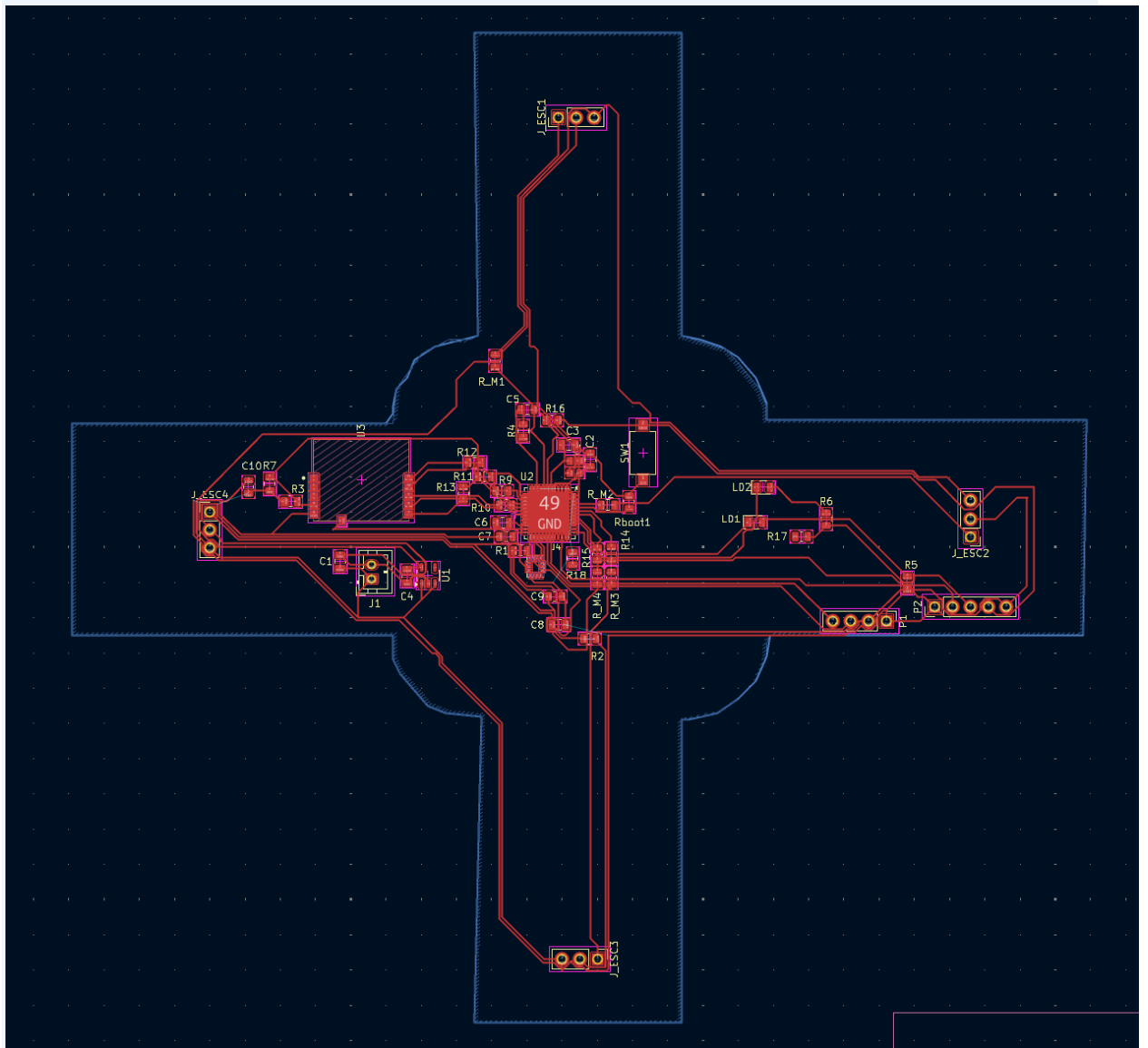
- ☐ Réassocier les empreintes aux symboles schématiques en utilisant leurs références schématiques
- ☐ Supprimer les empreintes sans symbole associé
- ☒ Remplacer les empreintes par celles spécifiées dans le schéma
- ☐ Supprimer/remplacer des empreintes même si elles sont verrouillées

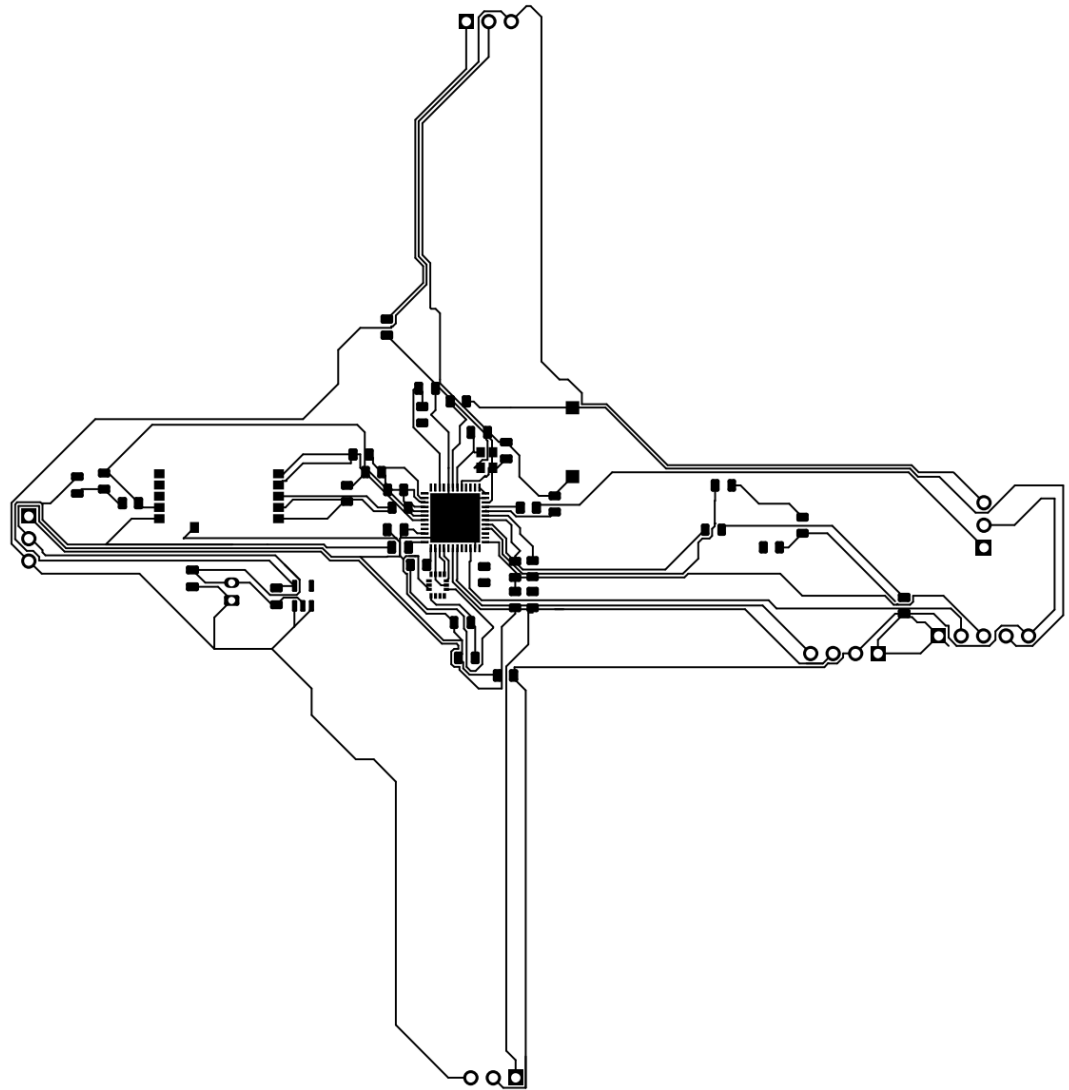
Modifications Appliquées au CI

Total warnings: 0, erreurs: 0.

Montrer: ☒ Tout ☒ Erreurs **0** ☒ Avertissements **0** ☒ Actions ☒ Infos Sauver...

Mise à jour PCB Fermer





IX. Conception de modèle en 3D

1. Paramètres Généraux de Conception

Les dimensions globales de la **structure** ont été contraintes par la taille d'une feuille A4, en prenant soin d'éviter d'utiliser les extrémités. Le centre de la structure a été privilégié pour offrir une grande surface dédiée au placement des pistes conductrices et des composants électroniques majeurs. La modélisation tridimensionnelle de l'ensemble des pièces a été réalisée à l'aide du logiciel Autodesk Fusion.

2. Conception du Moule du Châssis

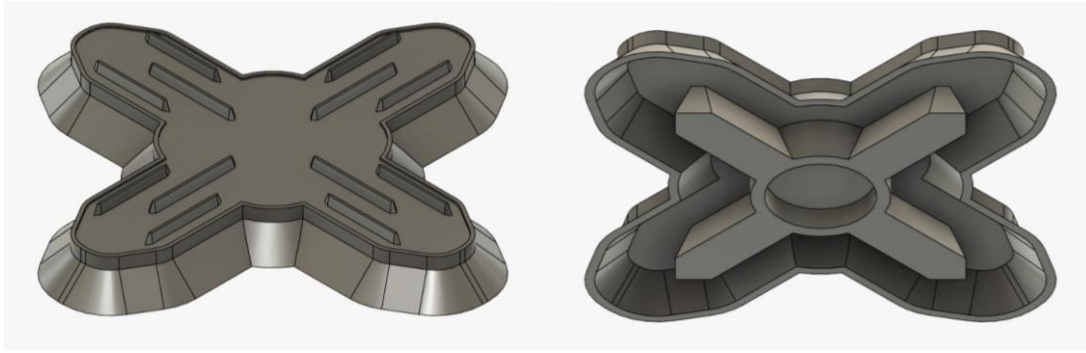
Le moule, défini par des dimensions de 16 cm par 16 cm, est destiné à la fabrication du châssis du drone. L'élaboration de cette pièce centrale a nécessité plusieurs itérations pour aboutir à un résultat fonctionnel.

Première itération (Échec) : La première version du moule s'est avérée défailante pour plusieurs raisons. Tout d'abord, la forme globale manquait d'épaisseur et de robustesse. L'absence d'éléments de renfort rendait la forme finale trop fragile. De plus, la présence d'un creux sous le moule a entraîné une déformation structurelle de celui-ci lors du processus de moulage. Enfin, les extrémités du moule étaient perpendiculaires au sol, ce qui a généré d'importantes difficultés lors de l'étape de démoulage pour séparer le moule du châssis en polymère.



Seconde itération (Version optimisée) : Pour pallier ces défauts, une nouvelle géométrie en forme de croix uniforme, dotée de quatre bras symétriques, a été adoptée. Les modifications suivantes ont été apportées :

- Les extrémités ont été repensées sous forme de pente avec un profil en trapèze afin de prévenir l'échappement d'air sous le moule pendant l'opération de moulage.
- Afin de rigidifier le futur châssis en polymère, de fins supports (sous forme de nervures) ont été ajoutés au niveau des bras du moule, à raison de trois supports de 3 cm x 2 mm pour chaque bras.
- Le moule a été imprimé en 3D au FabLab en utilisant du PLA. Ce matériau a été spécifiquement sélectionné pour sa grande accessibilité en impression 3D et pour sa robustesse, lui permettant de résister efficacement aux contraintes physiques du moulage.



1. Conception des Supports Moteurs et Protège-Hélices

La modélisation des supports destinés aux quatre moteurs a également suivi un processus d'amélioration continue. Suite à l'échec d'un premier concept de base, le design a été intégralement repensé.



Seconde itération : La nouvelle structure intègre un cylindre creux central d'un rayon de 5 mm et d'une hauteur de 2 cm. L'épaisseur du cerclage servant de protège-hélice a été fixée à 1,5 mm. La forme globale a été méticuleusement conçue pour garantir que les moteurs soient fermement fixés et sécurisés.

Des optimisations fonctionnelles ont été ajoutées à cette pièce :

- Une fente a été intégrée à l'extrémité du cylindre. Cette ouverture facilite grandement l'insertion du moteur dans son logement tout en permettant le passage aisé des fils électriques en vue de leur brasage avec le circuit principal.
- Le mécanisme d'attache de cet ensemble (associant le résultat du moulage et les quatre supports) repose sur un évidement sous le support des moteurs. Ce creusage permet d'y loger un écrou afin d'attacher et de verrouiller solidement les supports par-dessus le châssis.



X. Firmware STM32 — Architecture logicielle

1. Environnement de développement et base de code

Le firmware est développé sous “STM32CubeIDE”, l’environnement intégré de STMicroelectronics basé sur Eclipse. Le langage utilisé est le C++, avec la couche d’abstraction matérielle HAL (Hardware Abstraction Layer) fournie par ST, qui garantit la portabilité du code entre les différents microcontrôleurs STM32. Le firmware de référence STSW-FCU001, disponible en le téléchargeant de STMicroelectronics (lien en Annexe D), Le code source est utilisé comme point de départ et il est aussi organisé de manière modulaire afin de faciliter le développement, les tests et la maintenance.

Remarque :

Nous avions prévu de réaliser des essais de vol afin d’ajuster et modifier certains paramètres du contrôle du drone (PID, calibration, stabilité). Cependant, en raison de contraintes techniques et de temps, les tests de vol n’ont pas pu être réalisés. Par conséquent, le firmware n’a pas été modifié et les paramètres d’origine ont été conservés.

2. Structure du firmware

Le firmware est structuré en plusieurs modules indépendants. Chaque fichier source regroupe les fonctions liées à un périphérique ou à une fonctionnalité spécifique du drone. Cette organisation améliore la lisibilité du projet et permet de tester chaque bloc séparément.

Module	Fichier(s)	Rôle	Responsable de
Boucle principale	main.c	Orchestre tout le firmware	Init HAL, capteurs, BLE, timers — puis boucle à 800 Hz via TIM9
Moteurs	motor.c	Applique la puissance aux 4 moteurs	Écrit les valeurs PWM dans TIM4 (CCR1→CCR4), avec saturation min/max
Bus SPI	stm32f4xx_hal_spi.c	Transports donnés sur SPI	SPI1 → module BLE (SPBTLE-RF) / SPI2 → capteurs IMU
Driver IMU	LSM6DSL_ACC_GYRO_driver.c	Lit l'accéléromètre et le gyroscope	Lecture registres via SPI, conversion en mg et mdps
Pile Bluetooth	bluenrg_gap_aci.c + bluenrg_gatt_aci.c	Gère la communication BLE	GAP (connexion/appairage) + GATT (échange de données joystick)
PID	PID.c + flight_control.c	Stabilise le drone sur 3 axes	Calcule les corrections pitch/roll/yaw et les distribue aux 4 moteurs
Stabilisation AHRS	ahrs.c + quaternion.c	Estime l'orientation du drone	Fusion accel+gyro → quaternion → angles Euler (Mahony)

Télécommande	rc.c	Reçoit les ordres du pilote	Décode BLE (joydata) ou PWM → gAIL, gELE, gTHR, gRUD
Debug	debug.c	Sortie de débogage série	Envoie des messages via UART1 à 115200 bauds

3. Fonctionnement général du firmware

Cycle de fonctionnement :

Dès la mise sous tension, le microcontrôleur **STM32F401CCU6**, initialise le système via le fichier `main.c`. Il configure les bus SPI, les capteurs et le Timer TIM4 (cœur de la commande moteur). Une fois l'auto-test terminé, le firmware entre dans une boucle temps réel structurée en quatre phases.

Phase 1: Liaison Pilote (Bluetooth Low Energy)

Le pilotage repose sur le module **SPBTLE-RF** qui assure la communication avec l'application mobile.

- **Appairage** : Diffusion du nom DRN1120, connexion sécurisée par code PIN (123456).
- **Flux de données** : Réception 800 fois par seconde d'un tableau de 8 octets (`joydata[]`) contenant les positions des joysticks.
- **Interprétation** : Conversion immédiate des octets en variables de vol : Gaz (`gTHR`), Roulis (`gAIL`), Tangage (`gELE`) et Direction (`gRUD`).
- **Sécurité** : En cas de coupure du signal, un mécanisme "Fail-safe" arrête instantanément les moteurs.

Phase 2 : Acquisition de données (Capteurs)

Le drone surveille son état physique en temps réel via l'IMU (Inertial Measurement Unit).

- **Mesure** : Le capteur LSM6DSL (via SPI) fournit les accélérations et vitesses de rotation sur 3 axes à une fréquence de 160 Hz.

Phase 3 : Estimation de l'orientation (AHRS)

Les données brutes sont transformées en une orientation compréhensible (Attitude and Heading Reference System).

- **Fusion de données** : L'algorithme fusionne l'accéléromètre et le gyroscope pour créer un quaternion.
- **Conversion** : Ce quaternion est traduit en angles d'Euler (Pitch, Roll, Yaw) pour définir la position du drone dans l'espace.

Phase 4 : Boucle de contrôle (Double PID)

Le cerveau du drone corrige l'erreur entre la volonté du pilote et la réalité physique.

- **Boucle d'Angle (Externe)** : Calcule l'écart entre l'angle demandé par le smartphone et l'angle réel mesuré.
- **Boucle de Vitesse (Interne)** : Utilise un correcteur PID (Proportionnel, Intégral, Dérivé) pour stabiliser la rotation
- **Sortie Moteurs** : Le résultat est traduit en 4 signaux PWM à 500 Hz via le Timer TIM4, ajustant la puissance de chaque moteur DC pour réaliser la manœuvre.

4. Adaptation des paramètres

Après le premier vol :

Il est très probable que le drone ne soit pas parfaitement stable. Voici les ajustements à faire, dans l'ordre :

Remarque : Tous les paramètres de réglage se trouvent dans le fichier `flight_control.h`. Il suffit de modifier les valeurs, recompiler et reflasher la carte.

Réglage de la stabilité (gains PID) :

Le drone oscille ou tremble	Réduire <code>ROLL_PID_KP2</code> et <code>PITCH_PID_KP2</code> (valeur par défaut : 80)
Le drone est mou, il répond lentement	Augmenter progressivement <code>ROLL_PID_KP2</code> et <code>PITCH_PID_KP2</code> .
Le drone dérive lentement vers un côté	Augmenter légèrement <code>ROLL_PID_KI2</code> ou <code>PITCH_PID_KI2</code> (valeur par défaut : 80)
Des vibrations haute fréquence apparaissent	Réduire <code>ROLL_PID_KD2</code> / <code>PITCH_PID_KD2</code> (valeur par défaut : 10)
La rotation (yaw) est difficile à tenir	Ajuster <code>YAW_PID_KP2</code> (valeur par défaut : 900). Monter si le drone ne tient pas son cap, descendre s'il oscille en rotation.

Calibration mécanique :

Le drone penche toujours d'un côté même gaz à zéro	<code>X_AXIS_OFFSET</code> et <code>Y_AXIS_OFFSET</code>
Le gyroscope dérive (rotation lente sans commande)	<code>GYRO_OFFSET_X</code> , <code>GYRO_OFFSET_Y</code> , <code>GYRO_OFFSET_Z</code>
Un moteur est plus faible que les autres	<code>MOTOR_OFF1</code> à <code>MOTOR_OFF4</code>

XI. Tests à réaliser sur le prototype

5. LED : soudure et test

Avant d'assembler la carte FCU du drone, une activité pratique de soudure a été réalisée au Laboratoire Ampère de l'Université Lyon 1. L'objectif était de former l'équipe aux techniques de brasage manuel avant d'aborder les composants plus délicats de la carte de vol.

Circuit réalisé : un oscillateur astable à deux transistors bipolaires produisant un clignotement alterné de deux LED. Les composants assemblés sont des résistances, des condensateurs électrolytiques, deux transistors et deux LED.

Ce que l'activité a apporté :

- Maîtrise du fer à souder — température, durée, quantité de solder
- Lecture d'un schéma électronique avec documentation en anglais
- Identification et orientation correcte des composants avant brasage
- Compréhension du rôle des transistors en commutation

Résultat : le circuit fonctionne correctement. Les deux LED clignotent alternativement à la fréquence attendue. Les soudures sont propres, brillantes, sans pont entre pistes. Aucun composant endommagé.

Cette activité valide que l'équipe est prête techniquement pour l'assemblage du prototype FCU, dont la densité de composants et la sensibilité aux erreurs de soudure sont bien supérieures.

XII. Conclusion et perspectives

Ce projet nous a permis d'aborder la conception d'un drone non pas comme un simple assemblage de composants électroniques et mécaniques, mais comme un véritable système embarqué intégré, dans lequel chaque choix technique influence

directement la masse, la robustesse, l'autonomie et la fiabilité globale du prototype. À travers l'utilisation de la plastronique 3D et de la technologie IME, nous avons cherché à dépasser l'architecture classique des drones miniatures, généralement basée sur un circuit imprimé séparé du châssis, afin d'explorer une solution plus compacte, plus légère et mieux adaptée aux contraintes mécaniques d'un système volant.

Le travail réalisé a permis de définir une architecture complète du drone, structurée autour de sept blocs fonctionnels essentiels : le microcontrôleur STM32, l'alimentation, la communication Bluetooth, la centrale inertielle, les interfaces de programmation et de débogage, ainsi que les sorties de commande motrice. Cette décomposition a facilité la compréhension du système, la conception du schéma électronique sous KiCad, le choix des composants, ainsi que l'identification des contraintes propres à une réalisation en plastronique, notamment le routage monocouche, l'absence de plan de masse, les rayons de courbure minimaux des pistes et l'intégration des conducteurs dans une structure tridimensionnelle.

Sur le plan mécanique, la conception du châssis et des supports moteurs a montré l'importance d'une démarche itérative. Les premières versions ont mis en évidence plusieurs limites, notamment en matière de rigidité, de démoulage et de tenue structurelle. Ces difficultés ont conduit à une amélioration progressive de la géométrie, avec une structure en croix plus robuste, des bras renforcés et des supports moteurs mieux adaptés aux contraintes d'assemblage. Cette phase a confirmé qu'un drone plastronique ne peut pas être conçu en séparant la mécanique de l'électronique : les deux doivent être pensés simultanément.

Sur le plan électronique et logiciel, le projet a permis de s'appuyer sur une architecture proche de la carte de référence STEVAL-FCU001V2, tout en l'adaptant à une approche plus innovante. Le choix du STM32F401, du module Bluetooth BlueNRG-M0A, de l'IMU LSM6DSR et d'une commande motrice par signaux PWM fournit une base cohérente pour un prototype fonctionnel. Le firmware de référence constitue également un point de départ solide, notamment grâce à sa structure modulaire, sa boucle de contrôle, son algorithme de fusion de capteurs et son pilotage des moteurs.

Cependant, ce projet met aussi en évidence les limites d'une première version. Les essais de vol n'ayant pas pu être réalisés, la stabilité réelle du drone, le comportement dynamique du châssis, la tenue des pistes IME sous vibrations et l'autonomie effective restent à valider expérimentalement. Cette limite ne diminue pas la valeur du travail accompli ; elle précise simplement la frontière entre la conception validée sur le plan théorique et la validation complète en conditions réelles. Pour une suite de projet, les priorités seraient donc claires : finaliser l'assemblage, réaliser les tests électriques sous charge, mesurer la continuité des pistes après sollicitation mécanique, effectuer les premiers essais moteurs, puis procéder à une calibration progressive des paramètres PID.

À plus long terme, plusieurs améliorations pourraient être envisagées. Une optimisation du routage IME permettrait de réduire encore les pertes électriques et les risques d'interférences. L'intégration de moteurs brushless et d'ESC plus performants pourrait améliorer le rendement énergétique. L'ajout de capteurs complémentaires, comme un baromètre ou un capteur de distance, permettrait d'enrichir les fonctions de stabilisation et d'assistance au vol. Enfin, une étude comparative entre un drone à architecture PCB classique et un drone à architecture plastronique permettrait de

quantifier précisément les gains obtenus en masse, en volume, en robustesse et en temps d'assemblage.

En conclusion, ce projet constitue une première étape solide vers la réalisation d'un drone miniature à électronique intégrée dans la structure. Il a permis de démontrer la pertinence de la plastronique 3D pour repenser l'architecture des systèmes embarqués légers, tout en mettant en évidence les exigences techniques élevées qu'impose une telle approche. Au-delà du prototype lui-même, ce travail nous a permis de développer une vision plus complète de l'ingénierie système, où la mécanique, l'électronique, le logiciel et les contraintes de fabrication doivent être traités comme un ensemble indissociable. C'est précisément cette approche globale qui donne à ce projet sa valeur technique et son potentiel d'évolution.

XIII. Annexes

Annexe A — Schéma Kit de départ

- <https://fr.rs-online.com/web/p/outils-de-developpement-pour-microcontrolleurs/0279876>
- <https://fr.rs-online.com/web/p/outils-de-developpement-pour-microcontrolleurs/0279878>

Annexe B — Code firmware complet (texte copiable)

Extraits de code essentiels :

1. main.c — Boucle principale à 800 Hz (TIM9)

```

1 // Déclenché toutes les 1.25 ms par TIM9 → fréquence 800 Hz
2 if (tim9_event_flag == 1) {
3     tim9_event_flag = 0;
4
5     // Filtrage des données IMU
6     acc_ahrs.AXIS_X = 0;
7     acc_ahrs.AXIS_Y = 0;
8     acc_ahrs.AXIS_Z = 0;
9     gyro_ahrs.AXIS_X = 0;
10    gyro_ahrs.AXIS_Y = 0;
11    gyro_ahrs.AXIS_Z = 0;
12
13    for (i = 0; i < FIFO_Order; i++) {
14        acc_ahrs.AXIS_X += acc_ahrs_FIFO[i].AXIS_X;
15        gyro_ahrs.AXIS_X += gyro_ahrs_FIFO[i].AXIS_X;
16    }
17
18    acc_ahrs.AXIS_X *= FIFO_Order_Recip;
19    gyro_ahrs.AXIS_X *= FIFO_Order_Recip;
20
21    // Fusion des capteurs (AHRS Mahony)
22    ahrs_fusion_ag(&acc_ahrs, &gyro_ahrs, &ahrs);
23
24    // Quaternion → angles Euler
25    QuaternionToEuler(&ahrs.q, &euler_ahrs);
26
27    // Lecture joystick Bluetooth
28    gRUD = (joydata[2] - 128) * (-13);
29    gTHR = joydata[3] * 13;
30    gAIL = (joydata[4] - 128) * (-13);
31    gELE = (joydata[5] - 128) * 13;
32
33    // Armement moteurs
34    if ((gJoystick_status & 0x04) == 0x04) {
35        rc_enable_motor = 1;
36        fly_ready = 1;
37    } else {
38        rc_enable_motor = 0;
39        fly_ready = 0;
40    }
41
42    // Génération des angles cibles
43    GetTargetEulerAngle(&euler_rc, &euler_ahrs);
44
45    // PID externe
46    FlightControlPID_OuterLoop(&euler_rc_fil,
47                               &euler_ahrs,
48                               &ahrs,
49                               &pid);
50
51    // PID interne

```

```

52     if (rc_connection_flag && rc_enable_motor) {
53         FlightControlPID_innerLoop(&euler_rc_fil,
54                                     &gyro_rad,
55                                     &ahrs,
56                                     &pid,
57                                     &motor_pwm);
58     } else {
59         set_motor_pwm_zero(&motor_pwm);
60     }
61
62     // Sécurité moteurs
63     if (gTHR < MIN_THR)
64         set_motor_pwm_zero(&motor_pwm);
65
66     // Application PWM moteurs
67     set_motor_pwm(&motor_pwm);
68 }

```

2. motor.c — Commande PWM des 4 moteurs

```

1 void set_motor_pwm(MotorControlTypeDef *motor_pwm) {
2
3     // Moteur 1 – TIM4 canal 1
4     if (motor_pwm->motor1_pwm >= MOTOR_MAX_PWM_VALUE)
5         htim4.Instance->CCR1 = MOTOR_MAX_PWM_VALUE;
6
7     else if (motor_pwm->motor1_pwm <= MOTOR_MIN_PWM_VALUE)
8         htim4.Instance->CCR1 = MOTOR_MIN_PWM_VALUE;
9
10    else
11        htim4.Instance->CCR1 = (uint32_t)motor_pwm->motor1_pwm;
12
13    // Moteur 2 – TIM4 canal 2
14    htim4.Instance->CCR2 =
15        (uint32_t)motor_pwm->motor2_pwm;
16
17    // Moteur 3 – TIM4 canal 3
18    htim4.Instance->CCR3 =
19        (uint32_t)motor_pwm->motor3_pwm;
20
21    // Moteur 4 – TIM4 canal 4
22    htim4.Instance->CCR4 =
23        (uint32_t)motor_pwm->motor4_pwm;
24 }
25
26 // Arrêt d'urgence des moteurs
27 void set_motor_pwm_zero(MotorControlTypeDef *motor_pwm) {
28
29     motor_pwm->motor1_pwm = 0;
30     motor_pwm->motor2_pwm = 0;
31     motor_pwm->motor3_pwm = 0;
32     motor_pwm->motor4_pwm = 0;
33 }

```

3.LSM6DSL_ACC_GYRO_driver.c — Lecture registre IMU via SPI

```

1 // Lecture d'un registre du capteur IMU LSM6DSL via SPI
2 status_t LSM6DSL_ACC_GYRO_ReadReg(void *handle,
3                                     u8_t Reg,
4                                     u8_t *Data,
5                                     u16_t len) {
6
7     // Lecture des données du registre demandé
8     if (Sensor_IO_Read(handle, Reg, Data, len))
9         return MEMS_ERROR;
10
11     else
12         return MEMS_SUCCESS;
13 }
14
15
16 // Écriture d'un registre du capteur IMU
17 // Utilisé pour configurer l'accéléromètre et le gyroscope
18 status_t LSM6DSL_ACC_GYRO_WriteReg(void *handle,
19                                     u8_t Reg,
20                                     u8_t *Data,
21                                     u16_t len) {
22
23     // Envoi des données vers le registre du capteur
24     if (Sensor_IO_Write(handle, Reg, Data, len))
25         return MEMS_ERROR;
26
27     else
28         return MEMS_SUCCESS;
29 }

```



```

1 // Initialisation du module BLE en mode périphérique
2 // Le drone agit comme serveur Bluetooth Low Energy
3 tBleStatus aci_gap_init_IDB05A1(uint8_t role,
4                                  uint8_t privacy_enabled,
5                                  uint8_t device_name_char_len,
6                                  uint16_t *service_handle,
7                                  uint16_t *dev_name_char_handle,
8                                  uint16_t *appearance_char_handle) {
9
10     struct hci_request rq;
11     gap_init_cp_IDB05A1 cp;
12     gap_init_rp resp;

```

```

13
14     // Configuration du rôle BLE
15     cp.role = role;
16     cp.privacy_enabled = privacy_enabled;
17     cp.device_name_char_len =
18         device_name_char_len;
19
20     // Paramètres de la requête HCI
21     rq.ogf    = OGF_VENDOR_CMD;
22     rq.ocf    = OCF_GAP_INIT;
23     rq.cparam = &cp;
24     rq.clen   = sizeof(cp);
25     rq.rparam = &resp;
26     rq.rlen   = GAP_INIT_RP_SIZE;
27
28     // Envoi de la commande au module BLE
29     hci_send_req(&rq, FALSE);
30
31     return resp.status;
32 }

```

4. flight_control.c — PID boucle interne + calcul PWM moteurs

```

1 // Boucle PID interne : stabilisation du drone
2 void FlightControlPID_innerLoop(EulerAngleTypeDef *euler_rc,
3
4     Gyro_Rad *gyro_rad,
5
6     AHRS_State_TypeDef *ahrs,
7
8     P_PI_PIDControlTypeDef *pid,
9
10    MotorControlTypeDef *motor_pwm) {
11
12    float error, deriv;
13
14    dt_recip = 1 / pid->ts; // 800 Hz
15
16    // Axe X : Pitch
17    error = pid->x_s1 - gyro_rad->gx;
18
19    // Intégrale

```

```
15 pid_x_integ2 += error * pid->ts;
16
17 // Dérivée
18 deriv = (error - pid_x_pre_error2) * dt_recip;
19
20 // Filtre dérivé
21 deriv = pid_x_pre_deriv +
22     (deriv - pid_x_pre_deriv) * D_FILTER_COFF;
23
24 pid_x_pre_error2 = error;
25 pid_x_pre_deriv = deriv;
26
27 // Sortie PID
28 pid->x_s2 = pid->x_kp2 * error
29     + pid->x_ki2 * pid_x_integ2
30     + pid->x_kd2 * deriv;
31
32 // Saturation
33 if (pid->x_s2 > MAX_ADJ_AMOUNT)
34     pid->x_s2 = MAX_ADJ_AMOUNT;
35
36 if (pid->x_s2 < -MAX_ADJ_AMOUNT)
37     pid->x_s2 = -MAX_ADJ_AMOUNT;
38
39 // Conversion throttle → PWM
40 #ifdef MOTOR_DC
41 motor_thr = (int16_t)
42     (0.05f * (float)gTHR + 633.333f);
43 #endif
44
45 // Mixage moteurs quadrirotor
```

```

46 motor_pwm->motor1_pwm =
47     motor_thr - pid->x_s2 - pid->y_s2
48         + pid->z_s2 + MOTOR_OFF1;
49
50 motor_pwm->motor2_pwm =
51     motor_thr + pid->x_s2 - pid->y_s2
52         - pid->z_s2 + MOTOR_OFF2;
53
54 motor_pwm->motor3_pwm =
55     motor_thr + pid->x_s2 + pid->y_s2
56         + pid->z_s2 + MOTOR_OFF3;
57
58 motor_pwm->motor4_pwm =
59     motor_thr - pid->x_s2 + pid->y_s2
60         - pid->z_s2 + MOTOR_OFF4;
61 }

```

5.ahrs.c — Fusion Mahony (accéléromètre + gyroscope)

```

1 void ahrs_fusion_ag(AxesRaw_TypeDef_Float *acc, AxesRaw_TypeDef_Float *gyro,
2 Ahrs_State_TypeDef *ahrs) {
3 // Gain adaptatif : fort au démarrage (convergence rapide), normal en vol
4 ahrs_kp = (gTHR < MIN_THR) ? Ahrs_KP_BIG : Ahrs_KP_NORM; // 10.0 ou 0.4
5
6 // Convertir gyro de mdps → rad/s
7 gxf = gyro->AXIS_X * COE_MDPS_TO_RADPS; // 1.745e-5
8 gyf = gyro->AXIS_Y * COE_MDPS_TO_RADPS;
9 gzf = gyro->AXIS_Z * COE_MDPS_TO_RADPS;
10
11 // Normaliser l'accéléromètre
12 norm = invSqrt(axf*axf + ayf*ayf + azf*azf);
13 axf *= norm; ayf *= norm; azf *= norm;
14
15 // Direction estimée de la gravité depuis le quaternion courant
16 vx = 2*(q1*q3 - q0*q2);
17 vy = 2*(q0*q1 + q2*q3);
18 vz = q0*q0 - q1*q1 - q2*q2 + q3*q3;
19
20 // Erreur entre gravité mesurée et gravité estimée (produit vectoriel)
21 ex = (ayf*vz - azf*vy);
22 ey = (azf*vx - axf*vz);
23 ez = (axf*vy - ayf*vx);
24 // Correction intégrale (biais gyro)

```

```

25 exInt += AHRS_KI * ex;
26 eyInt += AHRS_KI * ey;
27 ezInt += AHRS_KI * ez;
28
29 // Intégration du quaternion avec correction Mahony
30 gxf += ahrs_kp * ex + exInt;
31 gyf += ahrs_kp * ey + eyInt;
32 gzf += ahrs_kp * ez + ezInt;
33
34 q0 += (-q1*gxf - q2*gyf - q3*gzf) * halfT;
35 q1 += ( q0*gxf + q2*gzf - q3*gyf) * halfT;
36 q2 += ( q0*gyf - q1*gzf + q3*gxf) * halfT;
37 q3 += ( q0*gzf + q1*gyf - q2*gxf) * halfT;
38
39 // Renormaliser le quaternion
40 norm = invSqrt(q0*q0 + q1*q1 + q2*q2 + q3*q3);
41 ahrs->q.q0 = q0 * norm;
42 // ...
43
44 }

```

6.debug.c — Sortie UART 115200

```

1 // Envoie un message formaté via UART1 (115200 bauds)
2 // Utilisation dans le code : PRINTF("pitch=%.2f roll=%.2f\n", p, r);
3 int myprintf(const char *format, ...) {
4     va_list arg;
5     va_start(arg, format);
6     char temp[255];
7     int len = vsnprintf(temp, 254, format, arg);
8     HAL_UART_Transmit(&huart1, (uint8_t *)temp, len, 1000);
9     return len;
10 }
11 // Activer avec #define DEBUG dans le projet CubeIDE

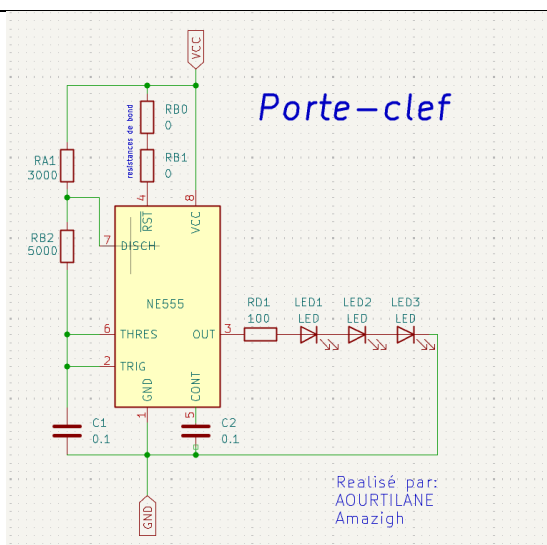
```

Annexe C — Circuits porte-clefs individuels

Pour chaque membre de l'équipe — ADJLIA, AOURTILANE, KANE, SEBA et GAOUA les éléments suivants sont joints dans cette annexe, identifiés par le nom du membre concerné : le schématique KiCad du circuit porte-clefs personnel, le schéma de placement et de routage PCB, une photographie du circuit après brasage des composants.

AOURTILANE

Schématique KiCad



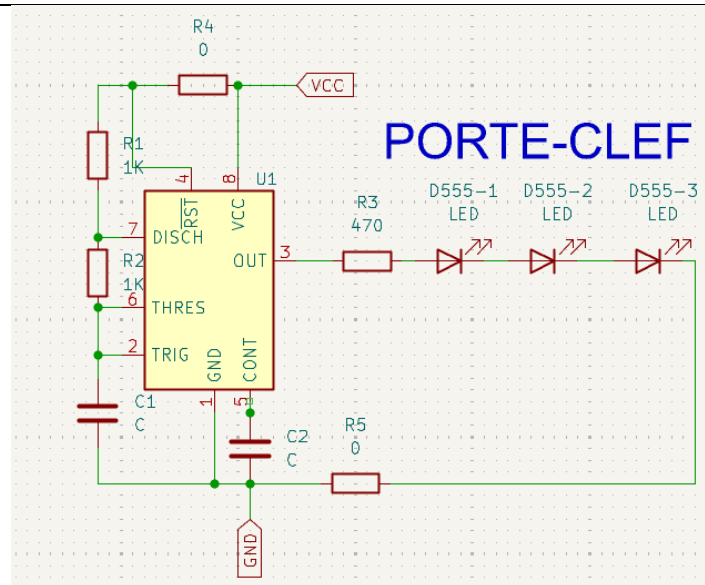
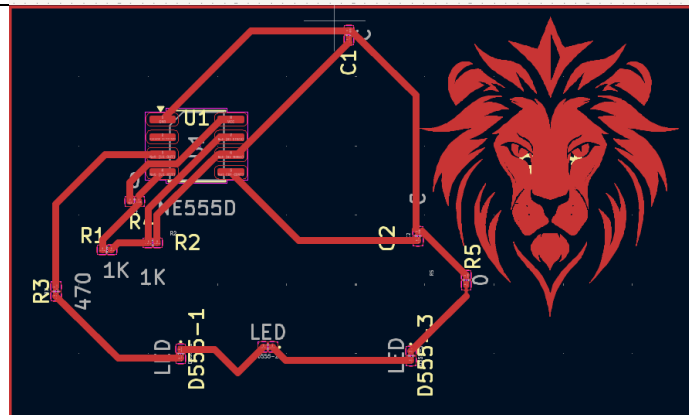
Routage PCB



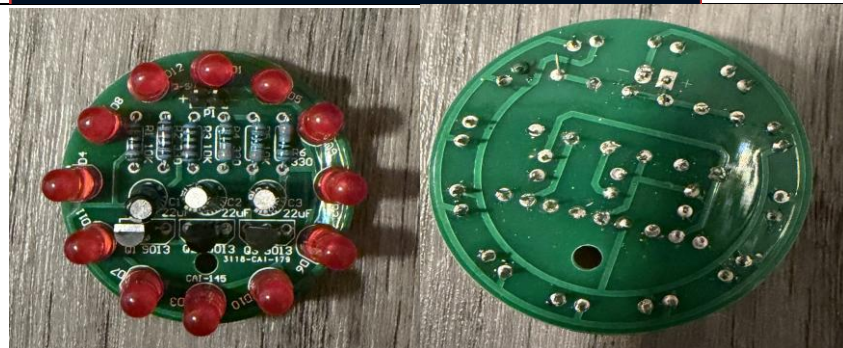
Brasage



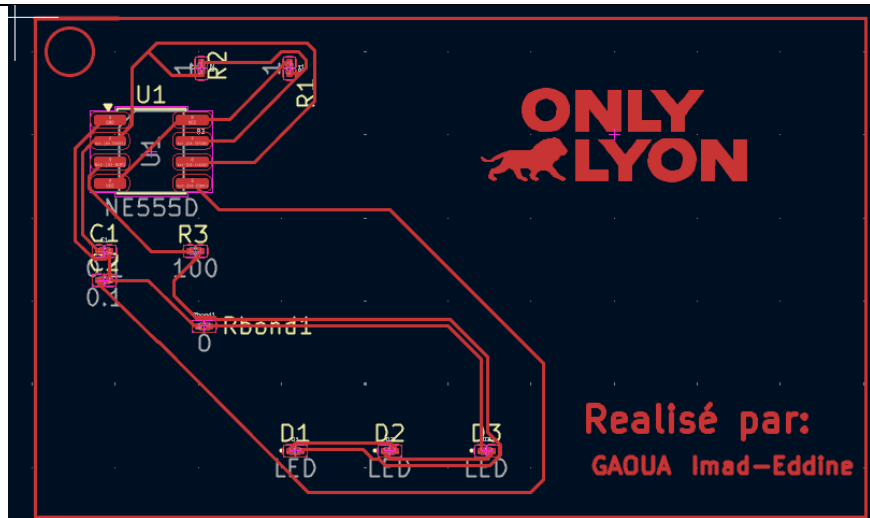
KANE

Schématique
KiCadRoutage
PCB

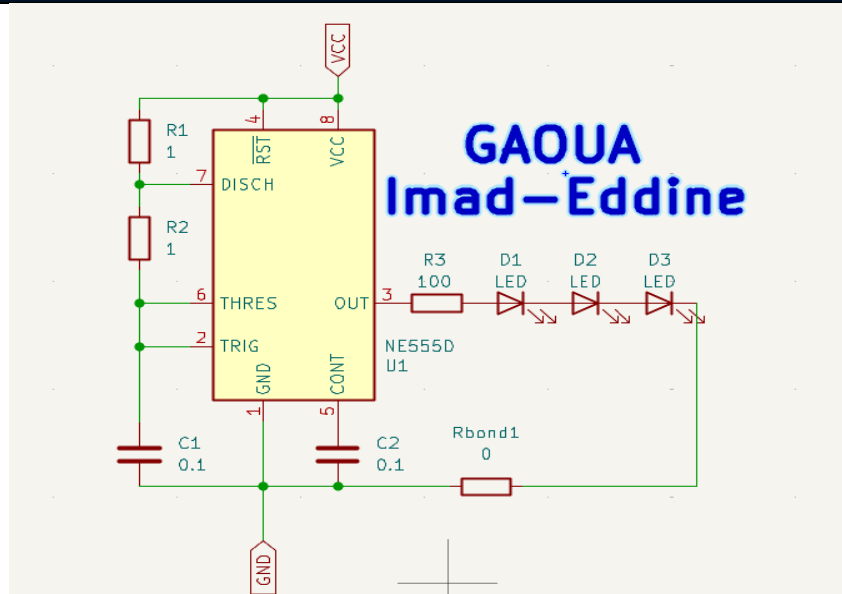
Brasage



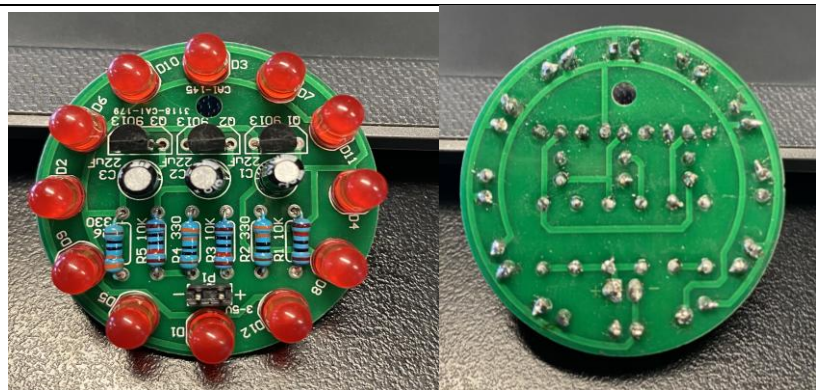
Schématique KiCad



Routage PCB

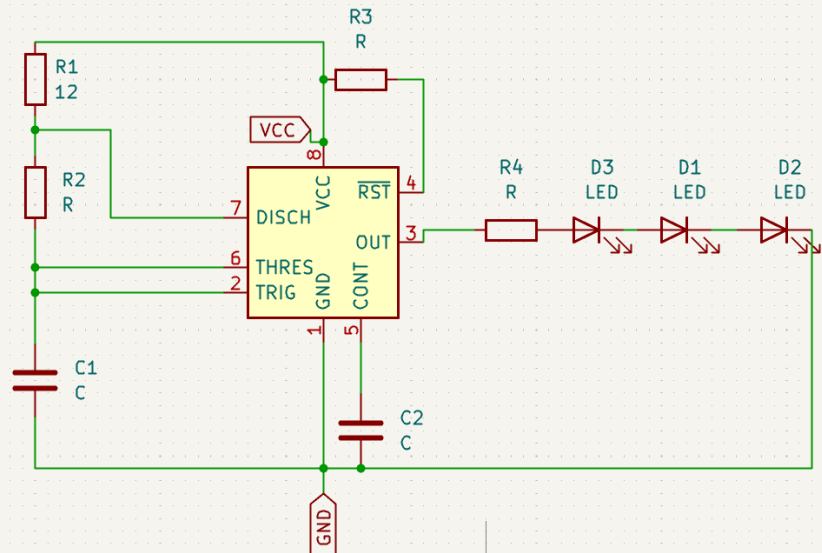


Brasage

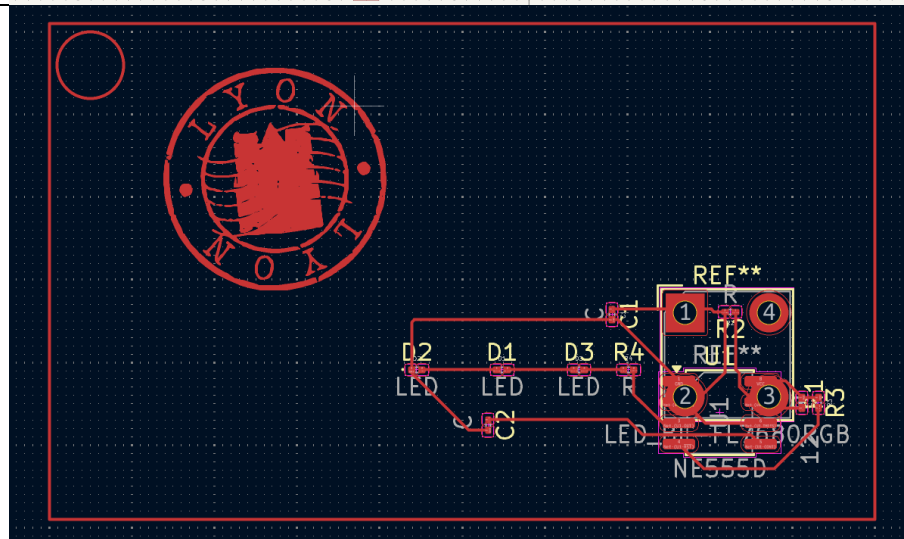


ADJLIA

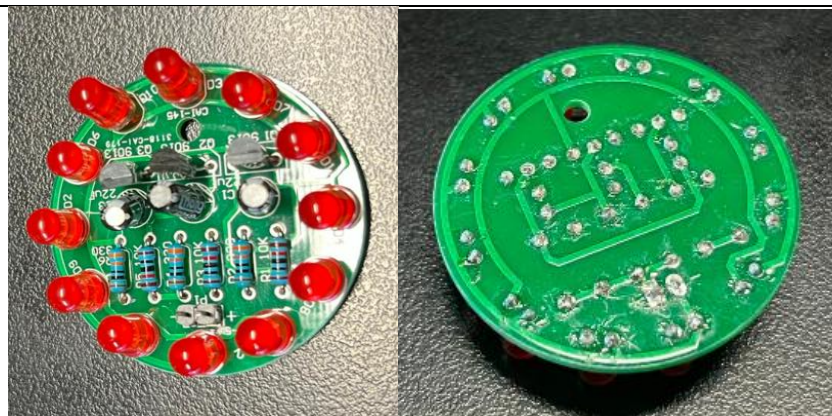
Schématique KiCad



Routage PCB

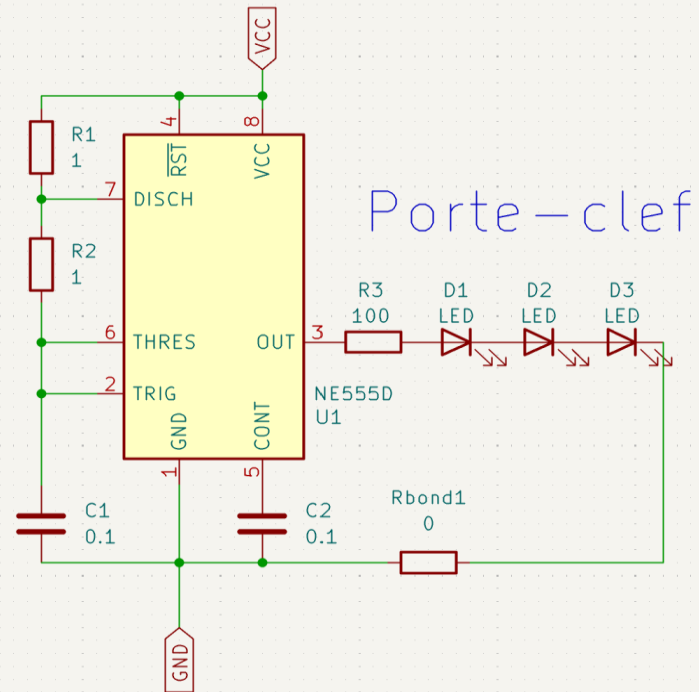


Brasage

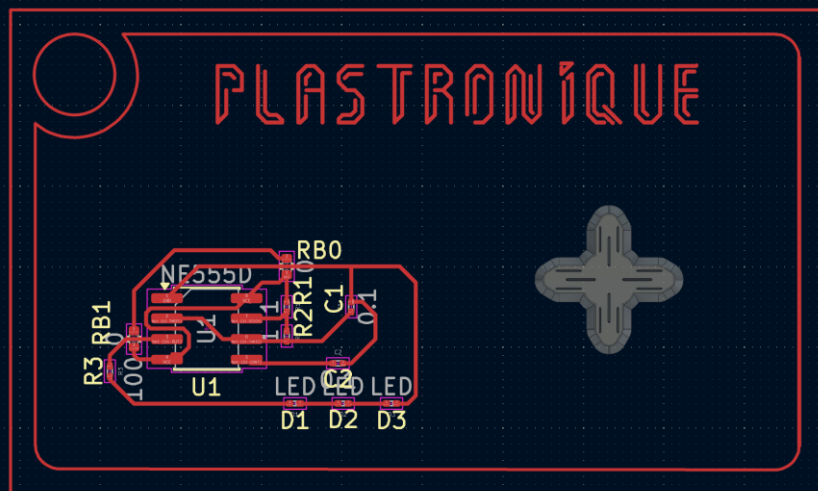


SEBA

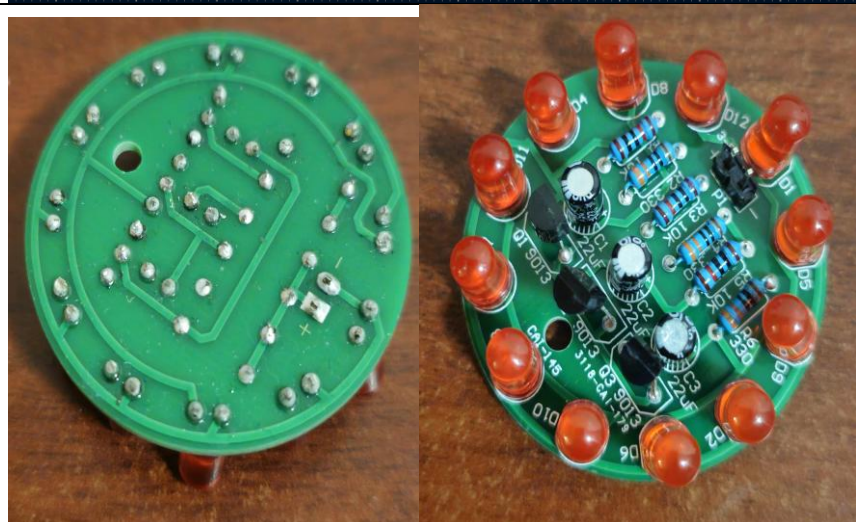
Schématique KiCad



Routage PCB



Brasage



Les documents techniques ayant servi de base à la conception sont les suivants.

- Le manuel utilisateur de la carte de référence est le document [ST UM2 958 Rev 2 \(juin 2024\)](#), intitulé [ST VAL-FCU001V2 User Manual](#).
- La fiche technique du module Bluetooth est le document [ST DS13023 Rev 5 \(mars 2021\)](#), [BlueNRG-M0 Datasheet](#).
- Le manuel de référence du microcontrôleur est [ST RM0368, STM32F401 Reference Manual](#).
- La fiche technique de la centrale inertielle est [ST DS10832, LSM6DSR Datasheet](#).
- La fiche technique du quartz est [NDK NX2520SA Crystal 16 MHz Datasheet](#).
- Source du code de la carte STM32 est [STSW-FC001](#).
- L'environnement de schématique utilisé est KiCad EDA 9.0.1, disponible sur <https://www.kicad.org>.
- Modélisation du Moule et Porte-Hélice avec AutoDesk Fusion, disponible sur <https://www.autodesk.com/products/fusion-360/overview>.
- Impression 3D Réalisé avec l'imprimante PRUSA MK4, <https://www.prusa3d.com/fr/produit/imprimante-3d-original-prusa-mk4s-5/#mk4vsmk4s>.